Task 1 (Easy): Convert `.then().catch()` to `async/await`

Objective: Convert a traditional Promise-based chain into an `async/await` function with `try/catch`.

Given Code (Promise Chain)

JavaScript

```
// Simulated helper function (Do not change)
function checkAge(age) {
  return new Promise((resolve, reject) => {
    if (age >= 18) {
      resolve("Access Granted: Welcome!");
    } else {
      reject("Access Denied: You must be at least 18.");
    }
  });
}

// Convert this execution code:
checkAge(20)
  .then((message) => {
    console.log(message);
  })
  .catch((error) => {
    console.log("Error:", error);
  });
```

Requirements

1. Create an `async` function named `verifyUserAge(age)`.
2. Inside `verifyUserAge`, call `checkAge(age)` using the `await` keyword.
3. Wrap the `await` call in a `try/catch` block:
    - **`try` block:** Log the resolved success message to the console.
    - **`catch` block:** Log `"Error:"` along with the rejection error message.
4. Test your function by calling `verifyUserAge(20)` and `verifyUserAge(15)`.

Task 2 (Average): Sequential Async Workflow with Error Handling

Objective: Execute multiple asynchronous operations in order using `await` and catch errors gracefully.

Helper Functions Provided

JavaScript

```
// Simulated database checks using delay
function verifyUser(username) {
  return new Promise((resolve, reject) => {
    setTimeout(() => {
      if (username === "admin") {
        resolve({ id: 101, username: "admin" });
      } else {
        reject("User not found in system.");
      }
    }, 1000);
  });
}

function getPermissions(user) {
  return new Promise((resolve, reject) => {
    setTimeout(() => {
      if (user.id === 101) {
        resolve(["READ", "WRITE", "DELETE"]);
      } else {
        reject("No permissions assigned.");
      }
    }, 1000);
  });
}
```

Requirements

1. Write an `async` function named `loginFlow(username)`.
2. Inside `loginFlow`, use `try/catch`:
    - **In `try`:**
        1. Await `verifyUser(username)` and store the returned user object in a variable.
        2. Log `"User Verified!"`.
        3. Pass that user object into `getPermissions(user)` and `await` the result.
        4. Log the retrieved permissions array to the console.
    - **In `catch`:**
        1. Catch any error that occurs during either step.
        2. Log `"Login Failed:"` followed by the error message.

3. Test the function twice:
    - Run `loginFlow("admin")` (should print success and permissions after 2 seconds).
    - Run `loginFlow("guest")` (should hit the catch block and print `"Login Failed: User not found in system."`).

Task 3 (Above Average): E-Commerce Checkout Pipeline

Objective: Handle a 3-step asynchronous sequence where any step can fail, use `await` sequentially, and implement a `finally` block for cleanup.

Helper Functions Provided

JavaScript

```
// Step 1: Check Item Inventory
function checkStock(item) {
  return new Promise((resolve, reject) => {
    setTimeout(() => {
      if (item === "Laptop") {
        resolve("In Stock");
      } else {
        reject("Item out of stock!");
      }
    }, 1000);
  });
}

// Step 2: Process Payment
function processPayment(amount) {
  return new Promise((resolve, reject) => {
    setTimeout(() => {
      if (amount <= 500) {
        resolve("Payment Approved");
      } else {
        reject("Card Declined: Insufficient funds.");
      }
    }, 1000);
  });
}

// Step 3: Generate Shipping Label
function createShippingLabel() {
  return new Promise((resolve) => {
```

```
    setTimeout(() => {
      resolve("Tracking ID: #TRK-99823");
    }, 1000);
  });
}
```

Requirements

1. Create an `async` function named `checkout(item, amount)`.
2. Wrap the sequence inside a `try / catch / finally` structure:
    - **In `try`:**
        1. Await `checkStock(item)`. Log `"Step 1 Passed: Item in stock"`.
        2. Await `processPayment(amount)`. Log `"Step 2 Passed: Payment successful"`.
        3. Await `createShippingLabel()`. Store the tracking ID and log `"Order Complete! " + trackingId`.
    - **In `catch`:**
        1. Catch any error thrown by any step in the pipeline.
        2. Log `"Checkout Aborted:"` followed by the error message.
    - **In `finally`:**
        1. Log `"Checkout session closed."` (This must execute regardless of whether the order succeeded or failed).
3. Test all edge cases:
    - `checkout("Laptop", 300)` (Should complete all steps successfully).
    - `checkout("Phone", 300)` (Should fail at Step 1: Out of stock).
    - `checkout("Laptop", 1000)` (Should pass Step 1, fail at Step 2: Insufficient funds).